

Angela Wei

Data 260

Homework 1

Section 0:

Personal Configuration and Domain

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

HTML:

CSS styling to make the form better organized

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Grocery Supply and Recall Notices</title>
<style>
  body {
    font-family: Arial, sans-serif;
    max-width: 600px;
    margin: 40px auto;
    padding: 20px;
    background-color: #f5f5f5;
    line-height: 1.6;
  }
</style>
</head>
```

Body of form with connection to JavaScript form.

```
<body>
<h1>Grocery Supply and Recall Notices</h1>
<h2>Enter Your Information</h2>
<form id = "domainForm">
  <label for = "sid4">Primary Field * :</label><br>
  <input type = "text" id = "sid4" placeholder= "Primary Field" required autofocus><br>

  <label for = "portBase">Secondary Field * :</label><br>
  <input type = "text" id = "portBase" placeholder = "Secondary Field" required><br>

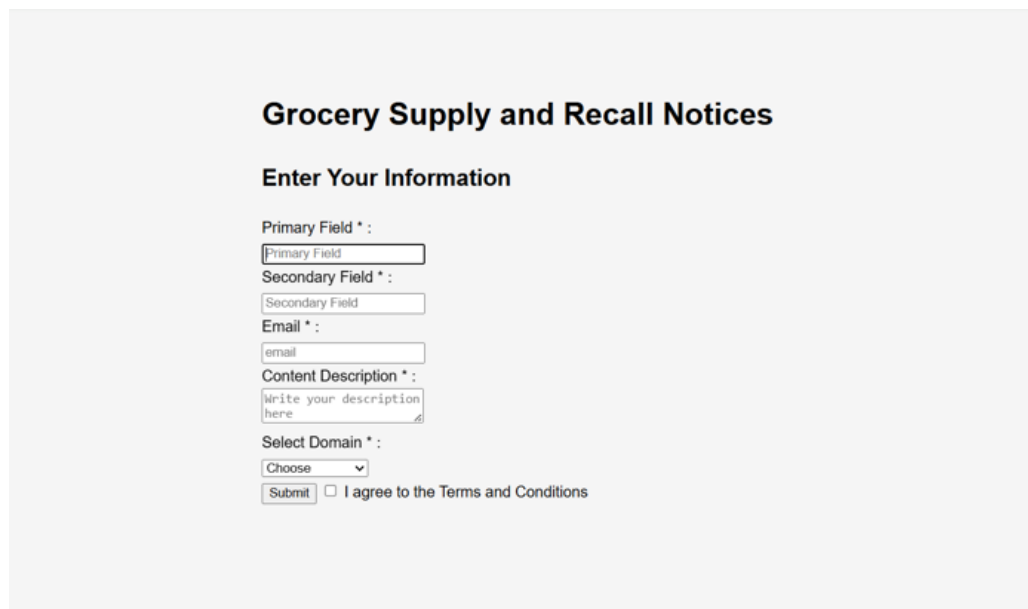
  <label for = "prefix">Email * :</label><br>
  <input type = "email" id = "prefix" placeholder = "email" required><br>

  <label for = "content">Content Description * :</label><br>
  <textarea id = "content" placeholder = "Write your description here" required></textarea><br>

  <label for = "domain_id">Select Domain * :</label><br>
  <select id = "domain_id" required>
    <option value = "" disabled selected>Choose</option>
    <option>Grocery Recall</option>
    <option>Supply Recall</option>
    <option>Food Safety</option>
    <option>Quality Control</option>
  </select><br>

  <button type = "submit">Submit</button>
  <input type = "checkbox" id = "acceptTerms" name = "acceptTerms" value = "Terms">
  <label for = "acceptTerms">I agree to the Terms and Conditions</label><br>
</form>
<script src = "HW1 - Angela Wei.js"></script>
```

Form as displayed on the web



The screenshot shows a web browser displaying a form titled "Grocery Supply and Recall Notices". Below the title is a subtitle "Enter Your Information". The form contains several input fields: "Primary Field *" (text input), "Secondary Field *" (text input), "Email *" (email input), "Content Description *" (text area), and "Select Domain *" (dropdown menu). At the bottom, there is a "Submit" button and a checkbox labeled "I agree to the Terms and Conditions".

JavaScript:

Arrow functions in JavaScript

```
// Arrow function to verify there are more than 25 characters
const validateFormSubmission = () => {
  const confirmInput = document.getElementById('content');
  const value = confirmInput ? confirmInput.value.trim() : '';
  if (value.length <= 25){
    alert('Must be longer than 25 characters');
    return false
  }
  return true
};

// Arrow function to check if the terms and conditions are validated
const validateTermsandConditions = () => {
  const checkTerms = document.getElementById('acceptTerms');
  const confirmCheck = checkTerms ? checkTerms.checked : false;
  if (!confirmCheck){
    alert('Please accept Terms and Conditions');
    return false
  }
  return true
};
```

Other form functionalities in Event Listener

```
// Event listener for all actions taken during form submission
document.getElementById('domainForm').addEventListener('submit', (event) => {
  const validateContent = validateFormSubmission();
  const validateTerms = validateTermsandConditions();

  // stop form from submitting if neither is validated
  if (!validateContent || !validateTerms) {
    event.preventDefault();
    return;
  }

  event.preventDefault();

  const getForm_Feedback = {
    primary : document.getElementById("sid4").value,
    secondary : document.getElementById("portBase").value,
    email : document.getElementById("prefix").value,
    content : document.getElementById("content").value,
    domain_info : document.getElementById("domain_id").value,
  };

  // cover the submitted information into JSON format
  const jsonOutput = JSON.stringify(getForm_Feedback);
  console.log("in JSON: ", jsonOutput);

  // extract primary field and email from parsed object
  const parse_output = JSON.parse(jsonOutput);
  const {primary, email} = parse_output;
  console.log("Primary Field:", primary);
  console.log("Email:", email);

  // get information on the date in which the form was submitted
  const parse_update = {
    ...parse_output,
    submissionDate : new Date().toISOString()
  };
});
```

JavaScript functionality when interacting with the form.

Grocery Supply and Recall Notices

Enter Your Information

Primary Field * :

Secondary Field * :

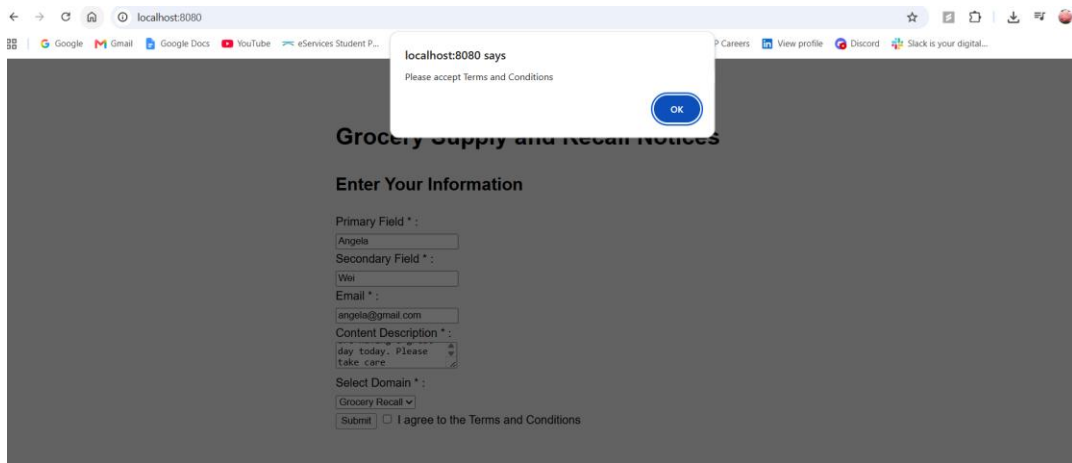
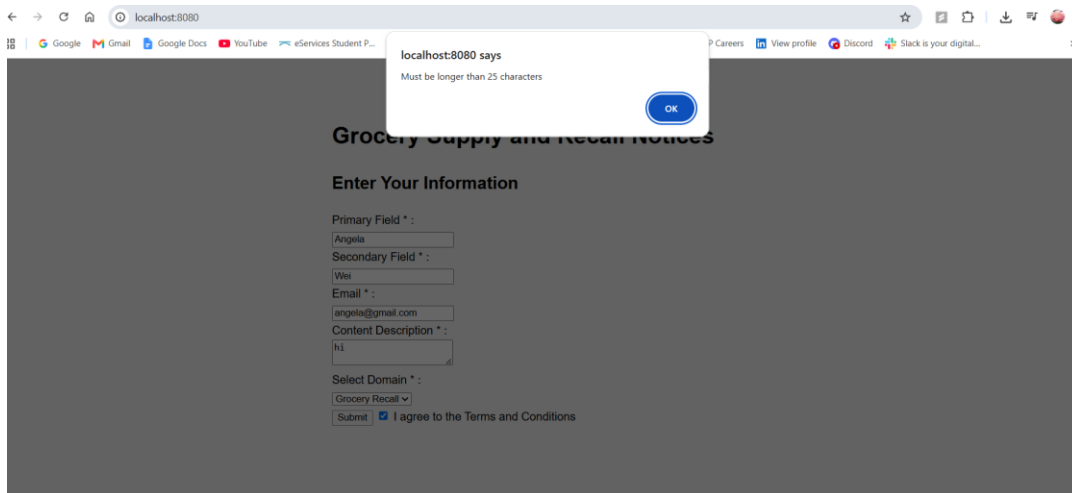
Email * :

Content Description * :

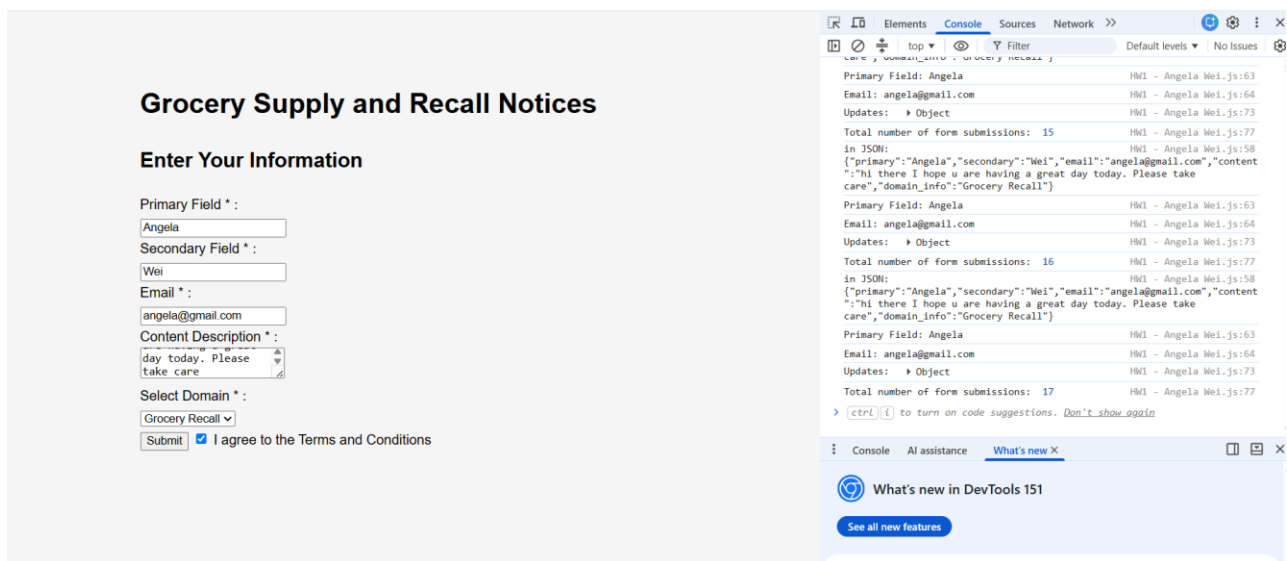


Please fill out this field.

☐ I agree to the Terms and Conditions



Form with JavaScript Console Logs next to it.



Docker Image and AWS ECS Service:

Building the Docker Image

```
PS C:\Users\wei.1\Desktop\AngelaWei\Angela-projects\Data260\Homework\my-web-app> docker build -t my-web-app .
[+] Building 13.4s (8/8) FINISHED
=> [internal] load build definition from Dockerfile                                0.0s
=> => transferring dockerfile: 344B                                              0.0s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine                 2.2s
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 10.4s
=> => resolve docker.io/library/nginx:alpine@sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 0.0s
=> => sha256:1f25fedd50aec27413031af3a4f8ee4effcc9d843f6a76e81bfa92245ac5c06 2.50kB / 2.50kB 0.0s
=> => sha256:7bc5ba2f958a043e123135f456af857350673b64eadccf811698239f3a53d6e6 12.32kB / 12.32kB 0.0s
=> => sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 3.85MB / 3.85MB 3.0s
=> => sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d 628B / 628B 0.3s
=> => sha256:db35bfc6b2951e7f8a72db5db120288c127ffaeeb4a6d4b95a26fead017d5913 10.33kB / 10.33kB 0.0s
=> => sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299 1.90MB / 1.90MB 1.9s
=> => sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61 955B / 955B 0.7s
=> => sha256:8d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb 404B / 404B 1.3s
=> => sha256:8cdf4e4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56 1.21kB / 1.21kB 1.6s
=> => sha256:da4dea1b00afb53e163ae330a511ecd6978371d99794c80f439b83b609eb9d66 1.40kB / 1.40kB 1.7s
=> => sha256:fb597529c91648bab2279939e01b745210875081852c73000aed882c000ff5de 20.50MB / 20.50MB 9.6s
=> => extracting sha256:55afa1ecc21d2bb5e5045f32dafee56272ffd89860bac26f6c32123439af26a4 0.2s
=> => extracting sha256:d94291c262619626bc7a27b61b090d85365e85c61f92ca49b190a26930121299 0.1s
=> => extracting sha256:0ea93572787841bba2f3921ece8dc2db925c22232a1388c766881ebe3bd3f67d 0.1s
=> => extracting sha256:1ed8e39a7434c8975957bb78530f191c92eecbd9c7309966192ba276a7c0ac61 0.0s
=> => extracting sha256:0d62d88506ba127db40e7c137e3a2dcf65e563709676828820b8c82277a6caeb 0.0s
=> => extracting sha256:8cdf4e4f237782ae36780c4634b1457e8d6efa6f929fc3e46b0a2074ba54acd56 0.0s
```

Docker Image running

Containers

Images

Volumes

Dev Environments

Extensions

Add Extensions

Containers

Give Feedback

A container packages up code and its dependencies so the application runs quickly and reliably from one computing environment to another. [Learn more](#)

Showing 1 items

Search

	NAME	IMAGE	STATUS	PORT(S)	STARTED	ACTIONS
☐	my-web-app-container c07e386b6231	my-web-app:latest	Running	8080	3 minutes ago	

HTML form running on localhost:8080

localhost:8080

Grocery Supply and Recall Notices

Enter Your Information

Primary Field * :

Secondary Field * :

Email * :

Content Description * :

Select Domain * :

☐ I agree to the Terms and Conditions

AWS ECS service running on Public IP address

Task lifecycle

- ✓ Created
September 1, 2026, 15:55 (UTC-7:00)
- ✓ Provisioning
 - ✓ Network interface attached
September 1, 2026, 15:55 (UTC-7:00)
- ✓ Pending
 - ✓ Pull started
September 1, 2026, 15:55 (UTC-7:00)
 - ✓ Pull completed
September 1, 2026, 15:55 (UTC-7:00) (1 second)
- ✓ Running
September 1, 2026, 15:55 (UTC-7:00)
Duration: 6 minutes, 19 seconds

Services (1) Info

Last updated September 1, 2026, 16:03 (UTC-7:00)

Manage tags Update Delete service Create

Filter services by value

Filter launch type Any launch type

Filter scheduling strategy Any scheduling strategy

Filter resource management type Any resource management type

< 1 > ⚙

Schedu...	L...	Task definition	Deployments and tasks	Last deployment	Last deplo...	Last deplo...
Replica	-	my-web-app-task:1	1/1 Tasks running	✓ Completed View	9 minutes ago	7 minutes ago

← → ↻ 🏠 Not secure 3.148.236.255 ☆ 📄 📁 📄 📄

🔍 Google 📧 Gmail 📄 Google Docs 📺 YouTube 🛒 eServices Student P... 🌐 one.SJSU 📄 Canvas Login - CSU... 🚀 CTI-Accelerate 📄 Login - COOP Careers 👤 View profile 🗨️ Discord 📄 Slack is your digital... »

Grocery Supply and Recall Notices

Enter Your Information

Primary Field * :

Secondary Field * :

Email * :

Content Description * :

Select Domain * :

☐ I agree to the Terms and Conditions

Part 2

AI Agent:

Planner and Review agents with finalization step

```
# Planner Agent
t0 = time.time()
a = planner.respond(transcript, task, args.title, args.content, args.strict)
t1 = time.time()
transcript.append({"role": "Planner", "content": json.dumps(a)})
print(f"\n--- Planner ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(a, indent=2)}")

# Reviewer Agent
t0 = time.time()
b = reviewer.respond(transcript, task, args.title, args.content, args.strict)
t1 = time.time()
transcript.append({"role": "Reviewer", "content": json.dumps(b)})
print(f"\n--- Reviewer ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(b, indent=2)}")

# Finalization
final = finalizer.respond(transcript, task, args.title, args.content, args.strict)
print(f"\n Finalized Output \n{json.dumps(final, indent=2)}")
```

Prompts and tasks given to agents

```
planner = SimpleAgent(name = "Planner",
                       system = ("Generate a draft with: "
                                "3 topical tags based on the given topic."
                                "A summary with 25 words maximum. No more than 25 words."
                                "tags format is [tag1, tag2, tag3]"
                                ),
                       model = llm)

reviewer = SimpleAgent(name = "Reviewer",
                       system = ("Review everything from the Planner's output"
                                "Planner should have generate 3 topical tags"
                                "The summary should have 25 words maximum. No more than 25 words"
                                "tags format is [tag1, tag2, tag3]"
                                ),
                       model = llm)

finalizer = SimpleAgent(name = "Finalizer",
                        system = ("Use Reviewer to finalize feedback"
                                "Ensure output has exactly 3 topical tags and summary has 25 words maximum"
                                "Output should be JSON with keys 'tags' and 'summary' "
                                "tags format is [tag1, tag2, tag3]"
                                ),
                        model = llm)

transcript = []
task = "Extract 3 topical tags and a summary of at most 25 words from provided title and content"
```

Output of Planner, Reviewer and Finalization.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\ai-agent> python ai_agents_demo.py --model llama3.2

--- Planner (62743 ms) ---
{
  "Grocery Supply and Recall Notices": [
    "recall",
    "grocery",
    "contamination"
  ],
  "Summary: 25 words": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores to ensure consumer safety and prevent health risks."
}

--- Reviewer (4460 ms) ---
{
  "recall": [
    "recall",
    "grocery",
    "contamination"
  ],
  "Summary": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores."
}

Finalized Output
{
  "tags": [
    "recall",
    "grocery",
    "contamination"
  ],
  "summary": "Groceries recalled due to contamination, foreign objects, or allergens. Products removed from stores."
}
```

Part 3

Non-Determinism Measurement:

```
llm = ChatOllama(  
    model=args.model,  
    temperature=0.0,  
    base_url=args.base_url,  
    num_ctx=2048,  
    format="json",  
)
```

When the temperature was set to 0.0, the only three tags that repeatedly showed up in every run were “grocery”, “recall” and “contamination”. There was no variety in the tags that showed up.

```
llm = ChatOllama(  
    model=args.model,  
    temperature=0.7,  
    base_url=args.base_url,  
    num_ctx=2048,  
    format="json",  
)
```

When the temperature was set to 0.7, “grocery”, “recall” and “contamination” still showed up, but there was a larger variety of tags, including but not limited to “safety”, “contamination”, “allergens”, as well as longer versions of the original three tags, such as “grocery supply”, or “recall notices”.

The temperature determines the creativity and predictability of an LLM’s response. This means the higher the temperature, the more variation there will be in responses from the LLM. A major LLM like ChatGPT or Gemini may give two users different answers to the same question due to having a high temperature set. This would be acceptable if the user was asking a question on creativity, but less acceptable when asking about factual information. In the latter case, the LLM could be giving out misinformation.

Part 4:

Model Client and Token Accounting:

Output after turn 3

```
Turn 3

{"tags": ["Syntax", "Logical"], "summary": "* \u2022 Line 1: if y == True: pass is an if statement with a conditional expression\n* \u2022 \u2022 The == operator is correctly used for comparison\n* \u2022 \u2022 The variable 'y' is correctly used in the condition\n* \u2022 \u2022 The pass statement is correctly used as a placeholder when no action is needed\n* \u2022 \u2022 The code is not handling potential errors or exceptions, and the condition is always true"}

Turn: 3, Input: 382

Output: 126

Total: 508

Stats after turn 3
Turn Number: 3
Input Tokens: 745
Output Tokens: 311
Total Turns: 1056
Conversation Length: 2035 characters
```

Output after turn 5

```
Turn 5

{"tags": ["Syntax", "Import"], "summary": "* \u2022 Line 1: import os, sys, json, time is an import statement\n* \u2022 \u2022 The imported modules are correctly specified\n* \u2022 \u2022 The import statement is correctly formatted\n* \u2022 \u2022 The code is not handling potential errors or exceptions, such as module not found\n* \u2022 \u2022 The import order could be improved for readability and maintainability"}

Turn: 5, Input: 689

Output: 113

Total: 802

Stats after turn 5
Turn Number: 5
Input Tokens: 1969
Output Tokens: 555
Total Turns: 2524
Conversation Length: 3167 characters
Turn Number: 5
Input Tokens: 1969
Output Tokens: 555
Total Turns: 2524
Conversation Length: 3167 characters
```

Initialization of LLM and token counts in model_client.py

```
class ModelAdapter:
    def __init__(self, model_name: str = "llama3.2", base_url: str = "http://localhost:11434", temperature: float = 0.0):
        self.llm = ChatOllama(
            model=model_name,
            temperature=temperature,
            base_url=base_url,
            num_ctx=2048,
            #format="json",
        )

        self.turns_count: int = 0
        self.input_tokens: int = 0
        self.output_tokens: int = 0
```

Retrieving information on token statistics and identify the JSON output in model_client.py

```
def stats_info(self, conversation_history: List[Dict[str, str]]) -> Dict[str, Any]:
    conversation_length = len(json.dumps(conversation_history))

    return {
        "total_turn_count": self.turns_count,
        "input_tokens": self.input_tokens,
        "output_tokens": self.output_tokens,
        "total_tokens": self.input_tokens + self.output_tokens,
        "length_of_conversation": conversation_length
    }

def _extract_json(self, raw_text: str):
    start = raw_text.find("{")
    end = raw_text.rfind("}")

    if start != -1 and end != -1 and end > start:
        return raw_text[start: end + 1]
    return raw_text
```

Code review prompts in HW1_client.py

```
prompts: List[str] = [
    "Turn 1: Review x = 3 + 12",
    "Turn 2: Review print('Hello' + name)",
    "Turn 3: Review if y == True: pass",
    "Turn 4: Review list = [1,2,3,4,5]",
    "Turn 5: Review import os, sys, json, time"
]

conversation_history: List[Dict[str, str]] = [{"role": "system", "content": system_instruction}]

for i, user_input in enumerate(prompts, start = 1):
    print(f"\n Turn {i}")
    conversation_history.append({"role": "user", "content": user_input})

    response_data, turns = adapter.complete(conversation_history)

    conversation_history.append({"role": "assistant", "content": json.dumps(response_data)})

    print(f"\n {json.dumps(response_data)}")
    print(f"\n Turn: {i}, Input: {turns['input_token_count']}")
    print(f"\n Output: {turns['output_token_count']}")
    print(f"\n Total: {turns['total_token_count']}")
```

Why is prior conversation context resent with every turn?

The conversation context needs to be resent for every turn because the LLM will not remember the prompt after each turn and will need to have it resent so it can give another output.

How is a system prompt different from a user message?

A system prompt focuses on how an LLM will behave overall in a conversation. A user message is a single prompt request in which the LLM will answer in a conversation.

Why do input tokens grow over a conversation?

Input tokens grow over the course of a conversation due to the number of inputs during each turn of a conversation increasing. More are being sent at once.

What eventually limits that growth?

The LLM hitting a technological limit such as a window cap or memory constraints in the computer hardware.

GitHub:

<https://github.com/Aywei2001/data260--SID4->